Smarter coding agents

Three research papers: managing context, remembering the repository, and planning the fix

01
Context as a Tool (CAT)

02
Repository Memory (RepoMem)

03
SGAgent

OVERVIEW

Three papers at a glance

01

Context as a Tool

Beihang University and others · Findings of ACL 2026

Proof of concept: the agent calls a tool that folds old history into a summary.

Problem: long agent runs overflow the context window.

02

Repository Memory

Ohio State University and Microsoft · ICLR 2026

Proof of concept: memory tools built from the repository's commit history.

Problem: agents start every bug from scratch.

03

SGAgent

NJUST and Nanjing University · arXiv preprint, May 2026

Proof of concept: a suggest stage that plans the fix first.

Problem: agents jump from locating a bug straight to patching it.

All three are tested on SWE-bench benchmarks: real GitHub issues in real Python repositories.

CAT: the agent manages its own context

THE PROOF OF CONCEPT

Context management becomes a tool the agent calls itself, like bash. A 32B parameter model is fine-tuned to learn when to do so.

THE PROBLEM

Agents append every step to the prompt. Over hundreds of rounds the context overflows and reasoning drifts.

1 · Fixed segment
System prompt and the task. Never compressed.

2 · Long-term memory
Summary of older steps, rewritten by the context tool.

3 · Working memory
The last k steps, kept word for word.

Training: CAT-Generator inserts context-tool calls into 20k recorded runs at natural milestones, then fine-tunes Qwen2.5-Coder-32B.

Source: Liu et al., Findings of ACL 2026, Sections 1 and 2

What the CAT results show

57.6%

of SWE-Bench-Verified issues solved (500 issues) by a 32B model

Agent, same 32B base model	Solved
ReAct, no context management	49.8%
Threshold-triggered compression	53.8%
SWE-Compressor (CAT), best	57.6%

Under 32k tokens

Average context stays flat after about 100 rounds instead of growing.

Up to 500 rounds

Accuracy keeps rising with a bigger budget; ReAct saturates near 60 rounds.

+3.8 points

over threshold compression, which uses the same summary scheme on a fixed trigger.

Source: Liu et al., Findings of ACL 2026, Table 2, Figures 4 and 5. All agents run on OpenHands.

RepoMem: memory built from commit history

Proof of concept: two memories from the commit history, exposed as four tools added to LocAgent.

Problem: code-localization agents start every issue from zero, with no knowledge of the repository.

EPISODIC MEMORY

What happened before

The 7,000 commits before the issue: messages, patches, linked issues.

Tools: **SearchCommit** (keyword search) and **ExamineCommit** (full diff).

SEMANTIC MEMORY

What each hot file does

LLM-written summaries of the 200 most frequently edited files.

Tools: **ViewSummary** (one file) and **SearchSummary** (search all).

Memory supplies the hypothesis; LocAgent's code-graph tools then verify it in the source.

Memory holds only commits made before the issue. Source: Wang et al., ICLR 2026, Sections 1, 3 and 4

What the RepoMem results show

File-level localization accuracy (%), SWE-bench-verified, GPT-4o

Method	Acc@1	Acc@3	Acc@5
CodeRankEmbed	29.6	45.1	54.3
Agentless	53.3	67.8	71.4
LocAgent (baseline)	64.8	70.4	71.6
RepoMem, best	68.6	74.5	76.5

+4.9 points

Acc@5 gain over LocAgent. On SWE-bench-live: 63.1 to 66.2.

+16.7 on pytest

Rich histories gain most: sphinx +11.4, django +7.4, matplotlib +5.9.

−13.1 points

Loss on repositories with under 10K commits (67.4 to 54.3).

Acc@k: top-k predicted files cover all files to fix. Source: Wang et al., ICLR 2026, Tables 1 and 2

PAPER 03 · SGAGENT · ARXIV PREPRINT, MAY 2026

SGAgent: plan the fix before the patch

Proof of concept: three agents in a localize-suggest-fix pipeline; the suggester writes a repair plan.

Problem: repair agents jump from where the bug is straight to a patch, so fixes come out shallow.

STEP 1 · LOCATE

Localizer

Searches the repository and proposes up to 5 candidate bug locations.

→

STEP 2 · SUGGEST · NEW

Suggester

Expands the context and writes a coordinated repair plan with risks.

→

STEP 3 · FIX

Fixer

Writes candidate patches; tests and voting select the final one.

All three share a knowledge graph of the repository: 3 entity types, 7 relations, 14 tools.

Source: Zhang et al., arXiv 2602.23647v2, Sections 1 and 3

What the SGAgent results show

51.3%

of SWE-Bench-Lite issues resolved with Claude-3.5-Sonnet (154 of 300)

Ablation on SWE-Bench-Lite	Resolved
Full SGAgent, best	51.3%
Without the knowledge graph	44.0%
Without the suggest stage	38.0%

60.7%

SWE-Bench-Lite with Claude-4-Sonnet as the base model (182 of 300).

65.4%

SWE-Bench-Verified with Claude-3.5-Sonnet (327 of 500).

48.0%

Java vulnerability repair, VUL4J and VJBench, with GPT-4o (24 of 50).

Average cost $1.48 per issue. Source: Zhang et al., arXiv 2602.23647v2, Tables 5, 7, 9, 11 and 12

COMPARISON

The three proofs of concept side by side

	CAT	RepoMem	SGAgent
Proof of concept	Context compression as a callable tool	Memory tools built from commit history	Suggest stage plus knowledge-graph tools
Problem	Context overflow in long runs	No memory of the repository	No plan between locate and fix
Training	Yes: fine-tunes a 32B model on 20k runs	None: tools added to LocAgent	None: train-free multi-agent framework
Task measured	Issue resolution	File-level code localization	Issue resolution
Headline result	57.6% on SWE-Bench-Verified	76.5% Acc@5 on SWE-bench-verified	51.3% on SWE-Bench-Lite
Gain shown	+7.8 points over plain ReAct	+4.9 points over LocAgent	+13.3 points from the suggest stage

Headline numbers use different tasks, models and benchmarks; do not compare them across papers.

CLOSING

Takeaways and limits

01

CAT

Takeaway: learned compression keeps long runs alive in a bounded context.

Limit: summaries are unverified; timing comes from heuristics.

02

RepoMem

Takeaway: the project's own history points the agent to the right files.

Limit: needs a rich commit history; thin ones lost 13.1 points.

03

SGAgent

Takeaway: a written repair plan turns bug locations into complete fixes.

Limit: results track model strength, from 32.7% to 60.7%.

CAT, RepoMem: authors' stated limits. SGAgent: this deck's reading. No paper tests the three together.